# Testbench Analysis

5-Class Cardiac Arrhythmia Classifier — RBF-SVM ASIC

Adam Handwerger · ECE410, Portland State University

2026-06-04

The verification strategy covers five levels, from bare RTL ports through the full Caravel SoC. Levels 1 and 2 target `svm_compute_core` directly; Level 3 verifies the `RAM_LATENCY` parameter added in m5; Levels 4 and 5 drive the complete `user_project_wrapper` through the Wishbone register interface. All 25 tests pass.

---

## Level 1 — Unit Tests (iverilog, Direct RTL Port)

**Location:** `m4/tb/` · **Simulator:** Icarus Verilog 13.0 · **Run:** `cd m4/tb && make all`

These 13 testbenches exercise individual FSM paths, error conditions, and datapath corner cases in isolation. They drive the core's RTL ports directly — no Wishbone bus, no Caravel wrapper — so a failure localizes immediately to the logic under test. Each test is self-contained and uses a small parameterization (FEATURE_DIM=16, NUM_SV=5–10) so that simulation completes in milliseconds and waveform inspection is practical.

### tb_top.sv — Full 5-Class Pipeline

Classifies one representative heartbeat per class (Normal, PVC, AFib, VT, SVT) using the trained SVM parameters from `tb_svm_params.svh`. The testbench programs gamma (0.25 -> 0x0100 in Q6.10), sets SV counts for all five classes, streams a 256-dimensional feature vector through the FIFO interface, services the SV RAM read requests from a precomputed hex file, and collects kernel outputs. After all SVs are processed, it applies the OVR decision function (weighted kernel sum + bias) and compares the predicted class against the expected label. Pass criterion is 4 of 5 correct classifications with no error flag asserted. This test is the primary smoke test for a new RTL build — if it fails, the issue is in the core pipeline rather than a corner case.

### tb_error_codes.sv — Fault Detection and Sticky Latch

Exercises all diagnostic error codes in sequence using two separate DUT instances. The main instance (FEATURE_DIM=16, NUM_SV=10, FIFO_DEPTH=256) tests ERR_SV_ZERO (code 0x1) by setting all SV counts to zero, ERR_SV_OVERFLOW (0x2) by loading more SVs than NUM_SV, and ERR_GAMMA_SAT (0x4) by writing a gamma value above the saturation threshold. A second instance (FIFO_DEPTH=4) tests ERR_FIFO_OVERFLOW (0x5) by streaming more words than the FIFO can hold. After triggering a fault, the testbench verifies that the `error_code` register holds its value for 50 idle cycles (sticky latch), then pulses `rst_n` and confirms that both `error` and `error_code` return to zero. This test directly validates the medical-device requirement that faults be persistently visible until the host explicitly resets the core.

### tb_backpressure.sv — Kernel-Ready Handshake

Tests the `kernel_valid` / `kernel_ready` flow-control handshake across three scenarios. Sub-test A runs with `kernel_ready` permanently asserted as the baseline. Sub-test B releases `kernel_ready` on the same clock edge that `kernel_valid` rises — verifying same- cycle acknowledgment. Sub-test C introduces a 3-cycle delay after `kernel_valid` rises before asserting `kernel_ready` — this was the scenario that exposed a bug

in an earlier RTL revision where kernel_valid pulsed for only one cycle and the FSM would stall permanently waiting for an acknowledgment it had already missed. The fix (a set/clear register that holds kernel_valid high until acknowledged) is verified here.

### tb_consecutive.sv — Back-to-Back Batch Resets

Runs two complete classification batches in succession without an intervening rst_n to confirm that the FSM and all internal counters reset cleanly between batches. After the first done fires, the testbench immediately pulses start again with a new num_samples value and a fresh feature stream. Key checks are that the sample_counter, class_counter, and sv_counter all return to zero at the batch boundary, that done fires exactly once per batch, and that no error flags carry over from one run to the next. This reflects normal wearable operation where 1000-beat batches are processed continuously.

### tb_dist_boundary.sv — Accumulator Saturation

Presents a worst-case feature pair where the input feature is 0x7FFF (+31.999 in Q6.10) and the SV feature is 0x8000 (–32.0 in Q6.10), maximizing the squared difference. For a 16-element FEATURE_DIM, this accumulates to a value that saturates the 20-bit accumulator at 0xFFFFF. The testbench verifies that the Horner LUT correctly maps this saturated distance to kernel_out = 0, which corresponds to exp(–∞) = 0 — the kernel value for two features that are as far apart as possible in the fixed-point space. The saturation is non-silent: ERR_DIST_OVERFLOW asserts if configured, so the host can distinguish a true distant feature pair from a numerical failure.

### tb_dist_zero.sv — Zero-Distance Kernel Identity

Sets every input feature equal to its corresponding SV feature (both 0x0400 = 1.0 in Q6.10) and verifies that the kernel output is exactly 1024 (= 1.0 in Q6.10). The expected computation trace is: diff = 0 for all dimensions, accumulator = 0, distance = 0, gamma × 0 = 0, LUT index = 0 -> lut_val = 1024, Horner residual = 0 -> correction = 1024, final result = (1024 × 1024) » 10 = 1024. This test is also the canonical check for the 2-cycle pipeline drain: if the FSM reads the accumulator one cycle too early, the last dimension is not yet included and the distance is nonzero, producing a kernel output below 1024 and failing the check.

### tb_gamma_zero.sv — Gamma Zero Advisory

Writes gamma = 0 to the parameter register and classifies a single heartbeat. When gamma = 0, the kernel evaluates to exp(0) = 1.0 for every SV regardless of distance, making all five class scores identical — the classifier returns a deterministic but meaningless result. The testbench verifies that the core asserts ERR_GAMMA_ZERO (code 0x6) as an advisory flag while still completing the batch without hanging. This ensures that a configuration error does not cause the FSM to stall or corrupt subsequent runs and that the host firmware has a diagnostic signal to detect and report the misconfiguration.

### tb_interface.sv — Port Signal Protocol

Checks the interface contract across 25 individual assertions covering reset state, register defaults, and protocol edge cases. Key checks include: all outputs (done, error, kernel_valid, qspi_ready) deasserted immediately after rst_n; gamma_reg reads back the default value (0x0100 = 0.25 in Q6.10); a start pulse while the FSM is not in IDLE is ignored and does not restart classification mid-batch; and a 2-sample batch terminates with exactly two sample_rdy pulses followed by exactly one done. This test is deliberately narrow and deterministic so that a failing assertion points to a specific RTL register or FSM transition.

### tb_min_sv.sv — Minimum SV Configuration

Configures one SV per class (sv_counts = [1,1,1,1,1], total 5 SVs) and classifies a single beat. This is the smallest valid non-trivial configuration and exercises the loop termination logic at its boundary condition. The testbench

verifies that exactly 5 kernel outputs are produced, that done fires once, and that all kernel outputs equal 1024 (because all SV features are loaded as zero and the input features are also zero, giving exp(0) = 1.0). A zero-SV or single-class edge case would trigger ERR_SV_ZERO and fail before reaching the kernel stage.

### tb_multi_heartbeat.sv — Three-Beat Loop-Back

Sets num_samples = 3 and streams three identical feature vectors in sequence, verifying that the FSM returns to LOAD_INPUT correctly after each beat and that done fires exactly once at the end of the third beat (not after each beat). The test also confirms that sample_rdy pulses three times — one per classified beat — with the correct class_out value each time. This is the primary test for the batch counter logic that governs how many heartbeats the core processes before halting.

### tb_param_write.sv — Gamma Shadow Register

Attempts to write a new gamma value mid-classification (during COMPUTE_DIST) using the param_write_en interface and verifies that the new value does not take effect until the next batch. An earlier RTL revision lacked a shadow register and the mid-compute write immediately changed the active gamma, causing the Horner LUT argument to shift partway through the distance accumulation and producing a corrupted kernel sum. The test confirms the fix: the core latches gamma at start into a shadow register, the live param_data path is disconnected during computation, and ERR_GAMMA_SAT fires if the written value exceeds the saturation threshold.

### tb_power.sv — Battery Fault Behavior

Tests the two-tier battery monitoring interface. ERR_LOW_BATTERY (code 0xA) is advisory: it fires when vbatt_warn asserts but allows the current classification to complete. The testbench checks that classification results are still valid under advisory fault conditions. ERR_POWER_FAIL (code 0xB) is a blocking fault: it fires when vbatt_ok deasserts and prevents start from launching a new batch. The 16 checks confirm that the advisory does not latch permanently (it clears when vbatt_warn deasserts), that the blocking fault overrides any advisory that is currently showing, and that the FSM returns to IDLE cleanly after a power fault rather than hanging in an intermediate state.

### tb_warmup.sv — Warmup Advisory Sequence

Verifies the two advisory codes related to the 100-beat warmup period. ERR_WARMING_UP (0x8) fires from a clean start and persists through beats 1–99 while the MCU accumulates enough history for the 10-beat mean morphology and 100-beat RR features to be reliable. ERR_INTERRUPTED (0x9) fires if rst_n asserts while the warmup counter is between 1 and 99, indicating that the previous warmup was cut short and a full uninterrupted 100-beat run is required before results can be trusted. Five sub-tests cover the normal warmup sequence, mid-warmup reset, real fault override (ERR_GAMMA_SAT latching sticky over an active advisory), advisory clearance at beat 100, and the re-warming-up condition after a reset that fires at count = 100. The timing note in the source confirms that the check waits two cycles after done because the advisory latches one cycle after the heartbeat counter increments.

---

## Level 2 — Integration Tests (cocotb, Direct RTL Port)

**Location:** m4/tb/ · **Simulator:** Icarus Verilog + cocotb 2.0.1 · **Run:** cd m4/tb && make cocotb

These 9 tests use Python coroutines to drive the same RTL ports as Level 1, enabling programmatic stimulus generation, floating-point reference computation, and result comparison in a single script. Cocotb tests run in simulated time alongside the RTL, yielding fine-grained control over timing that is tedious to express in pure SystemVerilog.

### test_reset_outputs

Confirms that every output port is in its reset state immediately after `rst_n` deasserts. The checked signals are `done`, `error`, `kernel_valid`, and `qspi_ready`, all of which must be low (or deasserted) within 120 ns of reset release. This test runs in under 10 clock cycles and acts as a gate for the remaining 8 tests — if output initialization is broken, no downstream test result is reliable.

### test_param_programming

Writes two parameter values — gamma and C — to the `param_write_en` / `param_addr` / `param_data` interface and reads them back via the `gamma_reg` and `c_reg` output ports. The test uses `to_fixed(0.25)` (= 0x0100) and `to_fixed(1.0)` (= 0x0400) to confirm that the Q6.10 encoding round-trips correctly through the register path. Any mismatch indicates a bit-width or sign-extension problem in the parameter latch.

### test_sv_counts_set

Programs an unequal but valid SV distribution — [60, 45, 55, 50, 40] totaling 250 SVs — and reads it back from the `num_sv_per_class` array. The checks confirm that each class entry stores independently and that the sum (250) does not trigger ERR_SV_OVERFLOW. This is the nominal training configuration from the m3 era (250 SVs, later doubled to 500 in m4/m5).

### test_sv_counts_unequal_stress

Applies an extreme asymmetric distribution [100, 10, 80, 40, 20] and confirms that the register file accepts it without error, even though one class (Normal) holds 5× more SVs than another (PVC). The test verifies that the SV counter loop handles non-uniform class sizes correctly and does not mis-index into the alpha table when the class boundaries are irregular.

### test_qspi_fifo_load *(name retained from m4 development; interface replaced by Wishbone + SRAM in m5)*

Streams a full 256-word feature vector through the QSPI interface (`qspi_valid` / `qspi_data` / `qspi_ready`) and confirms that the FIFO absorbs all 256 words without asserting `qspi_ready = 0` (no backpressure triggered). The simulation wall time is 10,420 ns — roughly 1042 clock cycles — reflecting one word per cycle throughput. This test was the initial vehicle for finding FIFO depth misconfigurations that would silently drop feature dimensions.

### test_qspi_backpressure *(name retained from m4 development; interface replaced by Wishbone + SRAM in m5)*

Deliberately overflows the FIFO by continuing to send `qspi_valid` after it is full, then verifies that `qspi_ready` deasserts to signal backpressure. Words presented while `qspi_ready` is low must not be written into the FIFO; the test confirms that the FIFO word count does not exceed FIFO_DEPTH and that no ERR_FIFO_OVERFLOW fires (because the host respected backpressure). The 168,000 ns simulation time reflects waiting for the core to drain the FIFO while new words are held off.

### test_default_gamma_fixed_point

Reads the `gamma_reg` output immediately after reset, before any `param_write_en` write, and confirms that it holds 0x0100 (= 0.25 in Q6.10). This verifies that the RTL parameter `DEFAULT_GAMMA = 0.25` is correctly encoded as the reset value of the register rather than being left as a floating-point literal that would synthesize incorrectly.

### test_full_pipeline_small_batch

Executes a complete single-sample pipeline end-to-end: programs parameters, sets 2 SVs per class (10 total), streams a feature vector, responds to all SV RAM read requests, collects kernel outputs, and waits for `done`. Pass criteria are that `done` fires exactly once in under 100,000 clock cycles and that `error` is not asserted. No accuracy check is performed here — this test is purely a pipeline liveness check that the FSM traverses all states (IDLE -> LOAD_INPUT -> COMPUTE_DIST -> COMPUTE_KERNEL -> WRITE_CLASS) without hanging.

### test_kernel_output_range

Runs the same small-batch pipeline as the previous test but adds a check on every kernel value produced: each `kernel_out` must lie in [0, 1024] in Q6.10 (corresponding to the mathematical range [0, 1] of the RBF kernel function $K(x, sv) = \exp(-\gamma d^2)$). A value outside this range indicates a fixed-point overflow, a sign error in the Horner polynomial, or an incorrect LUT entry. This test was written after an early RTL revision produced kernel values above 1024 due to a Horner coefficient sign flip.

---

## Level 3 — Feature Test: RAM_LATENCY (iverilog, Direct RTL Port)

**Location:** `m5/tb/` · **Simulator:** Icarus Verilog 13.0
**Run:** `iverilog -g2012 -DSIMULATION -o /tmp/svm_lat_tb.out ../rt1/compute_core.sv svm_ram_latency_tb.sv && /tmp/svm_lat_tb.out`

### svm_ram_latency_tb.sv — Wait-State Logic for Physical SRAM

This test was added in m5 specifically to verify the `RAM_LATENCY` parameter, which was introduced to support the IS61WV51216 asynchronous SRAM. In RTL cosimulation, the off-chip SRAM model is ideal (data valid on the same cycle as the read strobe, LAT=1). On physical silicon, the IS61WV51216 specifies a 10 ns access time; with PCB trace delays and flip-flop setup time, LAT=3 (three wait-state cycles) is required for reliable operation.

The testbench instantiates `svm_compute_core` with FEATURE_DIM=4, NUM_SV=5 (one SV per class), MAX_BATCH_SIZE=10, and RAM_LATENCY=3. The SRAM model is a 3-stage shift register on the address bus: `ram_rdata` presents the data from `addr_pipe[2]`, which holds the address that was presented three cycles earlier. This faithfully models a synchronous pipeline where address and data are both registered through three flip-flop stages.

Ten beats are classified with all features and SV values set to 0x0100 (= 1.0 in Q6.10), so every kernel output is exp(0) = 1024 and the expected class is 0 (Normal, highest accumulated score). The test confirms that `sample_rdy` fires exactly 10 times, `done` fires once at the end, and no sticky error code is asserted. ERR_WARMING_UP (advisory code 0x8) is expected for this 10-beat batch and is intentionally excluded from the failure condition. The measured throughput is 208 cycles per beat, which matches the analytical prediction: 5 SVs × (4 features × 3 wait-states + 18 kernel cycles) + FSM overhead ≈ 208 cycles.

---

## Level 4 — System Test (cocotb, Wishbone, Caravel Wrapper)

**Location:** `m5/tb/` · **Simulator:** Icarus Verilog + cocotb
**Run:** `PYTHONUNBUFFERED=1 make sim` (300 samples, ~96 min) or `COSIM_N_EVAL=25 make sim` (quick subset)

### tb_wb_cosim.py — 300-Sample Wishbone Cosimulation

This is the primary system-level verification. `tb_wb_cosim.py` acts as the host MCU and drives the full `user_project_wrapper` exclusively through the Wishbone B4 register interface — the same path that production firmware uses on Caravel silicon.

**Setup phase:** The script loads real MIT-BIH Arrhythmia Database ECG data using `wfdb` (PhysioNet library) and applies the same 80/20 stratified split as the training pipeline (`random_state=42`, 300 test samples, 60 per class). It trains an sklearn OVR-SVM with gamma=0.25 and C=1.0 in floating-point, extracts the 500 alpha coefficients and SV matrix in Q6.10, and writes them to a Python SRAM dictionary that models the off-chip IS61WV51216.

**Configuration:** The testbench writes to six Wishbone registers at base 0x3000_0000: `ALPHA_WR` (0x28) is written 500 times to load all alpha coefficients; `NUM_SV[0..4]` (0x10–0x20) sets 100 SVs per class; `NUM_SAMPLES` (0x0C) sets 300; `PARAM_WR` (0x24) encodes gamma = 0.25 as 0x0100; and `CONTROL` (0x04) fires the batch with `start = 1`.

**Classification loop:** Once started, the ASIC drives `ram_addr[18:0]` on GPIO[28:10] and `ram_ren` on GPIO[29] autonomously. The cocotb coroutine monitors `ram_ren`, looks up the address in the Python SRAM dictionary, and presents the 16-bit data word on `la_data_in[15:0]` on the next clock cycle (LAT=1 model). After each beat, `sample_rdy` (GPIO[3]) pulses and `class_out[2:0]` (GPIO[2:0]) holds the 3-bit class label. Results are accumulated in a list and written to `../sim/asic_preds.csv`.

**Result:** 293 of 300 samples classified correctly — 97.67% accuracy — with zero gap versus the sklearn float baseline. The 7 misclassified samples (4 VT->SVT, 3 SVT->VT) are shared with sklearn: both implementations misclassify the same beats because those beats lie on the RBF-SVM decision boundary where the margin is near zero, reflecting the intrinsic morphological ambiguity of VT and SVT on a single-lead ECG.

---

# Level 5 — Platform DV (Caravel DV, Wishbone, Full Management SoC RTL)

**Location:** `m5/tb/` · **Simulator:** Icarus Verilog + Caravel management SoC RTL
**Run:** `./dv_run.sh` (requires Caravel DV environment on Orca)

### svm_wb_test.c / dv_run.sh — RISC-V Firmware in Full-Chip Context

The Caravel DV framework compiles `svm_wb_test.c` to RISC-V machine code and runs it inside a full RTL simulation of the Caravel management SoC. Unlike Level 4, which drives Wishbone transactions directly from cocotb coroutines, Level 5 exercises the complete silicon path: the RISC-V processor fetches instructions from a model of on-chip flash, the C firmware executes Wishbone writes through the SoC's native bus fabric, the GPIO mux routes `io_out` signals to the user project's GPIO ports, and the Logic Analyzer interface drives `la_data_in`. This level verifies that the GPIO mux configuration, the `user_project_wrapper` port connectivity, and the Wishbone address decoding all function correctly in the full-chip context — not just in the isolated wrapper simulation of Level 4. Completion of the RTL simulation (all Wishbone register writes acknowledged without bus errors, GPIO transitions visible on the expected pins) constitutes a pass.

---

# Summary

| Level | Testbench(es) | Interface | Framework | Tests | Result |
|---|---|---|---|---|---|
| 1 — Unit | tb_top, tb_error_codes, tb_backpressure, tb_consecutive, tb_dist_boundary, tb_dist_zero, tb_gamma_zero, tb_interface, tb_min_sv, tb_multi_heartbeat, tb_param_write, tb_power, tb_warmup | Direct RTL | iverilog | 13 | **13/13 PASS** |
| 2 — Integration | test_reset_outputs, test_param_programming, test_sv_counts_set, test_sv_counts_unequal_stress, test_qspi_fifo_load, test_qspi_backpressure, test_default_gamma_fixed_point, test_full_pipeline_small_batch, test_kernel_output_range | Direct RTL | cocotb | 9 | **9/9 PASS** |
| 3 — Feature | svm_ram_latency_tb | Direct RTL | iverilog | 1 | **1/1 PASS** |
| 4 — System | tb_wb_cosim | Wishbone + wrapper | cocotb | 1 | **PASS — 97.67%** |
| 5 — Platform DV | svm_wb_test.c | Wishbone + SoC RTL | Caravel DV | 1 | **RTL sim complete** |
| **Total** | | | | **25** | **25/25 PASS** |

**Note on interface terminology in Levels 1 and 2:** The Level 1 and 2 tests were written when the core still used a QSPI/FIFO streaming interface, and their signal names reflect that era (e.g., `qspi_valid`, `qspi_ready`, `sv_ram_addr`). These tests drive `svm_compute_core` at its direct RTL ports, bypassing the Caravel wrapper entirely, so the internal port names have not changed even though the top-level communication strategy moved to Wishbone. The Wishbone interface lives in `top.sv` and translates incoming bus transactions into the same internal signals the unit tests drive directly. The Level 1 and 2 tests therefore remain valid for the final RTL. The only test that exercises the complete Wishbone path as silicon would see it is Level 4 (`tb_wb_cosim.py`), which passes at 97.67% accuracy.

---